ACID Properties

atomicity, consistency, isolation, durability & the WAL

Watch the 3-min video

In one line: ACID is the four promises a transaction makes: all or nothing, valid state to valid state, no peeking at half-done work, and committed means kept. Durability comes from the write-ahead log, flushed to disk before the commit is acknowledged.

1 The problem: a transfer that must not half-happen

Alice sends Bob $100. The database must do two things: subtract 100 from Alice and add 100 to Bob. If the server crashes between them, the money must not vanish, and it must not appear out of thin air. A **transaction** groups both changes into one unit, and four guarantees, **ACID**, make that unit safe.

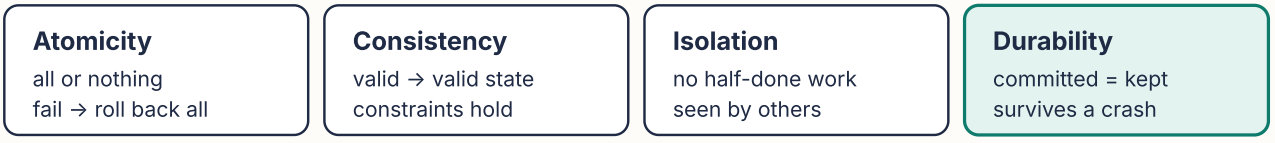


Figure 1. The four ACID guarantees. Durability is the one the write-ahead log provides.

2 The four letters

- **Atomicity: all or nothing.** Both updates happen, or neither does. If anything fails before the commit, the database **rolls back** every change in the transaction.
- **Consistency: valid state to valid state.** Declared rules are enforced: a `CHECK (balance >= 0)`, foreign keys, unique keys. But consistency is partly your job. The database enforces the rules you declare; your code must keep the business invariants, such as “the total amount of money stays the same”.
- **Isolation: no half-finished work is visible.** Ideally, concurrent transactions behave as if they ran one at a time. In practice, databases offer **isolation levels** that trade strictness for speed (next lesson).
- **Durability: committed means kept.** Once the database says “committed”, the data survives a crash or a power cut.

3 Worked example: the transfer

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE name = 'alice';
UPDATE accounts SET balance = balance + 100 WHERE name = 'bob';
COMMIT;

-- table has: CHECK (balance >= 0)
```

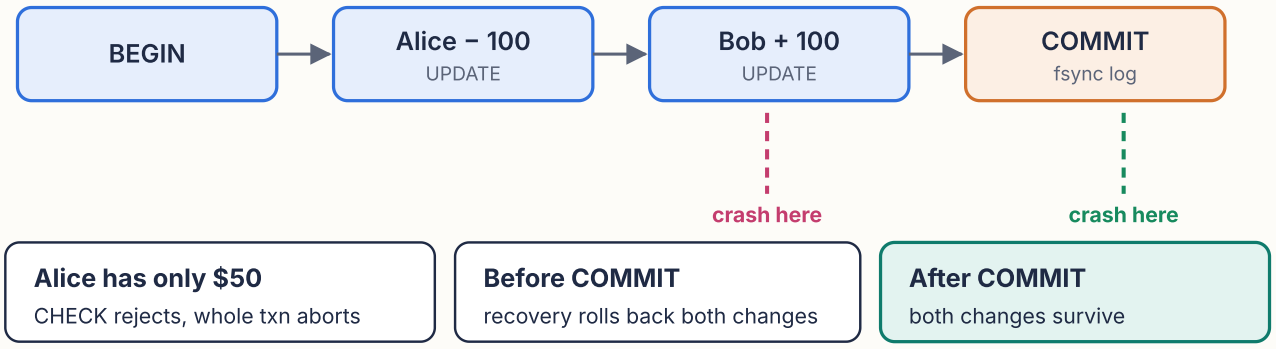


Figure 2. Three outcomes. A failed constraint or a crash before COMMIT leaves no trace; a crash after COMMIT loses nothing.

WORKED EXAMPLE

- **Alice has only $50.** Her balance would become −50, the check constraint rejects it, and the whole transaction aborts. Bob gets nothing either (atomicity + consistency).
- **Crash before COMMIT.** On restart, both changes are rolled back. Alice still has her money.
- **Crash after COMMIT.** Both changes survive, because the log records were already on disk (durability).

4 How durability works: the write-ahead log

Writing every changed data page to disk at commit time would be slow: pages are scattered all over the disk. Instead the database keeps a **write-ahead log** (WAL), an append-only file of change records. The rule is in the name: **log first, data later**.

1. Before changing a data page, the database appends a record of the change to the log.
2. On commit, it flushes the log to disk with `fsync`, and only then tells the client "committed". Sequential appends are fast.
3. The changed data pages stay in memory and are written later, in the background (a **checkpoint**).
4. After a crash, **recovery replays the log**: it redoes committed changes that never reached the data files, and discards uncommitted ones.

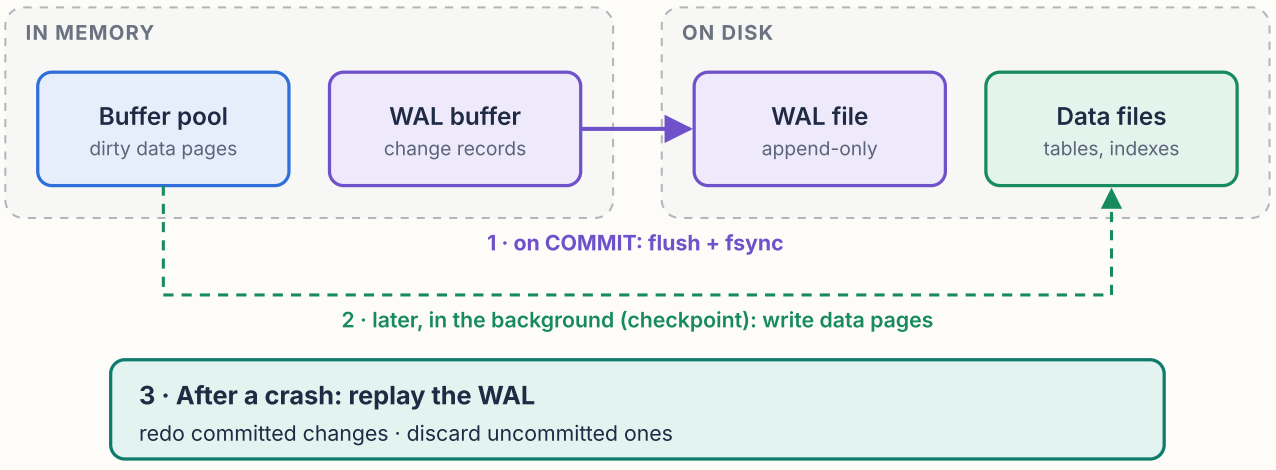


Figure 3. The WAL turns many random page writes at commit into one sequential log flush. The data files catch up later.

5 Durability has knobs

Setting	What it does	Trade-off
Synchronous commit	Commit waits for the WAL fsync on the primary, and optionally for a replica to confirm it.	With a replica: losing a whole server loses nothing. Costs a round trip.
Group commit	Batches many concurrent transactions into one fsync.	Much higher throughput, same safety.
Asynchronous commit	Acknowledges before the fsync completes.	Faster, but a crash can lose the last few milliseconds of commits.

All of this is about **one** database. Atomicity across several databases or services needs other tools: **two-phase commit** or **sagas**, which come later in the course.

COMMON MISTAKES

- Assuming the database alone keeps your business rules. It only enforces the constraints you declare: add CHECK, UNIQUE and foreign keys.
- Running a multi-step change without a transaction (autocommit per statement), so a crash leaves it half done.
- Confusing ACID consistency (constraints hold) with CAP consistency (every replica returns the latest write).
- Turning on asynchronous commit for money or orders, where losing the last commits is not acceptable.

6 Key terms

Transaction: a group of changes applied as one unit

Invariant: a business rule your code must keep

Checkpoint: writing dirty pages so less log must be replayed

Commit / rollback: make permanent / undo every change

WAL: append-only log written before data pages

Group commit: many commits share one fsync

Constraint: a rule the database enforces (CHECK, UNIQUE, FK)

fsync: force buffered data onto the physical disk

Recovery: replay the log after a crash

7 Interview questions

Q1. Explain each letter of ACID. BASIC

Atomicity: all of a transaction's changes happen, or none do. Consistency: transactions keep data valid, respecting constraints and invariants. Isolation: concurrent transactions don't see each other's partial work. Durability: committed data survives crashes.

Q2. How does a database guarantee durability? BASIC

With a write-ahead log. Changes are appended to the log and the log is fsynced before the commit is acknowledged. Data pages are written later. After a crash, the log is replayed: committed changes are redone, uncommitted ones discarded.

Q3. Why not just write the data pages at commit time? BASIC

A transaction may touch pages scattered across the disk; writing them all means many random writes per commit. The WAL needs one sequential append and one fsync, and group commit shares that fsync across many transactions.

Q4. A transfer of $100 runs, but Alice has only $50. What happens? SCENARIO

Her balance would go to −50, the CHECK (balance >= 0) constraint fails, and the whole transaction aborts. Bob's +100 is rolled back too. Neither row changes.

Q5. The server loses power 1 ms after the client saw “committed”. Is the data safe? SCENARIO

Yes, with synchronous commit: the WAL was fsynced before the acknowledgement, so recovery replays it. With asynchronous commit, the last few milliseconds of commits can be lost.

Q6. You need zero data loss even if the whole primary server is destroyed. What do you configure? SCENARIO

Synchronous replication: commit waits until at least one replica has the WAL record too. It adds a network round trip (often 1–2 ms in the same region) to every commit.

Q7. Is “consistency” in ACID the same as in CAP? TRICKY

No. ACID consistency means declared constraints and invariants hold after each transaction. CAP consistency means every read sees the latest write across replicas (linearizability).

Q8. Who keeps the invariant “total money is unchanged by a transfer”? TRICKY

Your code. The database can't know that rule; it only enforces declared constraints. Your transaction must debit and credit the same amount, and atomicity makes sure both or neither apply.

Next: SD-E-009 · Isolation Levels & Write Skew backendroadmap.com